Credit Card Default

In [25]:
```
#This project is on credit card default
```

In [93]:
```
!date
```

```
Sat 13 May 2023 12:51:54 PM EDT
```

In [94]:
```
from IPython import display
display.Image("/home/harlohutch77/pictures/cc.gif",height='330',width='500')
```

Out[94]:
```
<IPython.core.display.Image object>
```

In [4]:
```
# imports from libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import cufflinks as cf
```

In [6]:
```
# reading the csv file using a path to the file
ccdf = pd.read_csv('/home/harlohutch77/python/python_master/UCI_Credit_Card.csv')
ccdf
```

Out[6]:

	ID	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	...	BILL_AMT4	BILL_AMT5	BILL_AMT6
0	1	20000.0	2	2	1	24	2	2	-1	-1	...	0.0	0.0	0.0
1	2	120000.0	2	2	2	26	-1	2	0	0	...	3272.0	3455.0	3261.0
2	3	90000.0	2	2	2	34	0	0	0	0	...	14331.0	14948.0	15549.0
3	4	50000.0	2	2	1	37	0	0	0	0	...	28314.0	28959.0	29547.0
4	5	50000.0	1	2	1	57	-1	0	-1	0	...	20940.0	19146.0	19131.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
29995	29996	220000.0	1	3	1	39	0	0	0	0	...	88004.0	31237.0	15980.0
29996	29997	150000.0	1	3	2	43	-1	-1	-1	-1	...	8979.0	5190.0	0.0
29997	29998	30000.0	1	2	2	37	4	3	2	-1	...	20878.0	20582.0	19357.0
29998	29999	80000.0	1	3	1	41	1	-1	0	0	...	52774.0	11855.0	48944.0
29999	30000	50000.0	1	2	1	46	0	0	0	0	...	36535.0	32428.0	15313.0

30000 rows × 25 columns

In [7]:
```
ccdf.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 25 columns):
 #   Column                      Non-Null Count  Dtype  
---  ------                      --------------  -----  
 0   ID                          30000 non-null  int64  
 1   LIMIT_BAL                   30000 non-null  float64
 2   SEX                         30000 non-null  int64  
 3   EDUCATION                   30000 non-null  int64  
 4   MARRIAGE                    30000 non-null  int64  
 5   AGE                         30000 non-null  int64  
 6   PAY_0                       30000 non-null  int64  
 7   PAY_2                       30000 non-null  int64  
 8   PAY_3                       30000 non-null  int64  
 9   PAY_4                       30000 non-null  int64  
 10  PAY_5                       30000 non-null  int64  
 11  PAY_6                       30000 non-null  int64  
 12  BILL_AMT1                   30000 non-null  float64
 13  BILL_AMT2                   30000 non-null  float64
 14  BILL_AMT3                   30000 non-null  float64
 15  BILL_AMT4                   30000 non-null  float64
 16  BILL_AMT5                   30000 non-null  float64
 17  BILL_AMT6                   30000 non-null  float64
 18  PAY_AMT1                    30000 non-null  float64
 19  PAY_AMT2                    30000 non-null  float64
 20  PAY_AMT3                    30000 non-null  float64
 21  PAY_AMT4                    30000 non-null  float64
 22  PAY_AMT5                    30000 non-null  float64
 23  PAY_AMT6                    30000 non-null  float64
 24  default.payment.next.month  30000 non-null  int64  
dtypes: float64(13), int64(12)
memory usage: 5.7 MB
```

Statistics on Credit Card Default dataframe - csv file

In [93]:
```
#here is the description of the dataset starting with count to max
ccdf.describe().transpose()
```

Out[93]:

	count	mean	std	min	25%	50%	75%	max
ID	30000.0	15000.500000	8660.398374	1.0	7500.75	15000.5	22500.25	30000.0
LIMIT_BAL	30000.0	167484.322667	129747.661567	10000.0	50000.00	140000.0	240000.00	1000000.0
SEX	30000.0	1.603733	0.489129	1.0	1.00	2.0	2.00	2.0
EDUCATION	30000.0	1.853133	0.790349	0.0	1.00	2.0	2.00	6.0
MARRIAGE	30000.0	1.551867	0.521970	0.0	1.00	2.0	2.00	3.0
AGE	30000.0	35.485500	9.217904	21.0	28.00	34.0	41.00	79.0
PAY_0	30000.0	-0.016700	1.123802	-2.0	-1.00	0.0	0.00	8.0
PAY_2	30000.0	-0.133767	1.197186	-2.0	-1.00	0.0	0.00	8.0
PAY_3	30000.0	-0.166200	1.196868	-2.0	-1.00	0.0	0.00	8.0
PAY_4	30000.0	-0.220667	1.169139	-2.0	-1.00	0.0	0.00	8.0
PAY_5	30000.0	-0.266200	1.133187	-2.0	-1.00	0.0	0.00	8.0
PAY_6	30000.0	-0.291100	1.149988	-2.0	-1.00	0.0	0.00	8.0
BILL_AMT1	30000.0	51223.330900	73635.860576	-165580.0	3558.75	22381.5	67091.00	964511.0
BILL_AMT2	30000.0	49179.075167	71173.768783	-69777.0	2984.75	21200.0	64006.25	983931.0
BILL_AMT3	30000.0	47013.154800	69349.387427	-157264.0	2666.25	20088.5	60164.75	1664089.0
BILL_AMT4	30000.0	43262.948967	64332.856134	-170000.0	2326.75	19052.0	54506.00	891586.0
BILL_AMT5	30000.0	40311.400967	60797.155770	-81334.0	1763.00	18104.5	50190.50	927171.0
BILL_AMT6	30000.0	38871.760400	59554.107537	-339603.0	1256.00	17071.0	49198.25	961664.0
PAY_AMT1	30000.0	5663.580500	16563.280354	0.0	1000.00	2100.0	5006.00	873552.0
PAY_AMT2	30000.0	5921.163500	23040.870402	0.0	833.00	2009.0	5000.00	1684259.0
PAY_AMT3	30000.0	5225.681500	17606.961470	0.0	390.00	1800.0	4505.00	896040.0
PAY_AMT4	30000.0	4826.076867	15666.159744	0.0	296.00	1500.0	4013.25	621000.0
PAY_AMT5	30000.0	4799.387633	15278.305679	0.0	252.50	1500.0	4031.50	426529.0
PAY_AMT6	30000.0	5215.502567	17777.465775	0.0	117.75	1500.0	4000.00	528666.0
default.payment.next.month	30000.0	0.221200	0.415062	0.0	0.00	0.0	0.00	1.0

In [ ]:

In [62]:
```
ccdf.mean()
```

Out[62]:
```
ID                             15000.500000
LIMIT_BAL                     167484.322667
SEX                                1.603733
EDUCATION                          1.853133
MARRIAGE                           1.551867
AGE                               35.485500
PAY_0                             -0.016700
PAY_2                             -0.133767
PAY_3                             -0.166200
PAY_4                             -0.220667
PAY_5                             -0.266200
PAY_6                             -0.291100
BILL_AMT1                      51223.330900
BILL_AMT2                      49179.075167
BILL_AMT3                      47013.154800
BILL_AMT4                      43262.948967
BILL_AMT5                      40311.400967
BILL_AMT6                      38871.760400
PAY_AMT1                        5663.580500
PAY_AMT2                        5921.163500
PAY_AMT3                        5225.681500
PAY_AMT4                        4826.076867
PAY_AMT5                        4799.387633
PAY_AMT6                        5215.502567
default.payment.next.month         0.221200
dtype: float64
```

In [66]:
```
ccdf.std()
```

Out[66]:
```
ID                              8660.398374
LIMIT_BAL                     129747.661567
SEX                                0.489129
EDUCATION                          0.790349
MARRIAGE                           0.521970
AGE                                9.217904
PAY_0                              1.123802
PAY_2                              1.197186
PAY_3                              1.196868
PAY_4                              1.169139
PAY_5                              1.133187
PAY_6                              1.149988
BILL_AMT1                      73635.860576
BILL_AMT2                      71173.768783
BILL_AMT3                      69349.387427
BILL_AMT4                      64332.856134
BILL_AMT5                      60797.155770
BILL_AMT6                      59554.107537
PAY_AMT1                       16563.280354
PAY_AMT2                       23040.870402
PAY_AMT3                       17606.961470
PAY_AMT4                       15666.159744
PAY_AMT5                       15278.305679
PAY_AMT6                       17777.465775
default.payment.next.month         0.415062
dtype: float64
```

In [ ]:
```
#different ways to bring up your data - many ways with csv
#ccdf[ccdf['ID']==29999][['AGE','SEX']]
#ccdf[ccdf['default.payment.next.month']==1]['ID']
```

In [72]:
```
ccdf[['LIMIT_BAL','SEX','AGE','EDUCATION','MARRIAGE','default.payment.next.month']].count()
```

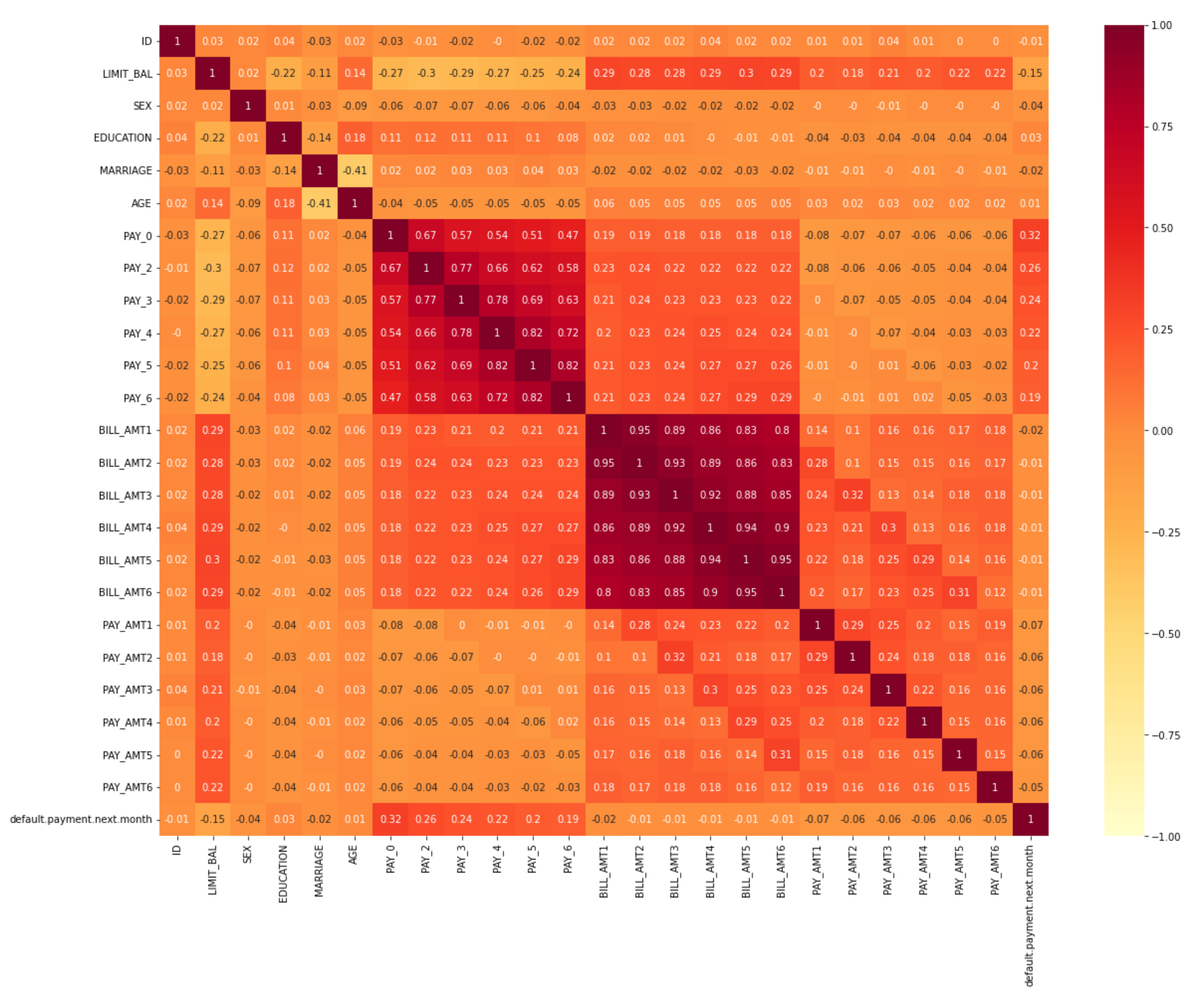
Out[72]:
```
LIMIT_BAL                     30000
SEX                           30000
AGE                           30000
EDUCATION                     30000
MARRIAGE                      30000
default.payment.next.month    30000
dtype: int64
```

In [5]:
```
#how many people are going to be in default next month 6636
ccdf[ccdf['default.payment.next.month']==1].count()
```

Out[5]:
```
ID                            6636
LIMIT_BAL                     6636
SEX                           6636
EDUCATION                     6636
MARRIAGE                      6636
AGE                           6636
PAY_0                         6636
PAY_2                         6636
PAY_3                         6636
PAY_4                         6636
PAY_5                         6636
PAY_6                         6636
BILL_AMT1                     6636
BILL_AMT2                     6636
BILL_AMT3                     6636
BILL_AMT4                     6636
BILL_AMT5                     6636
BILL_AMT6                     6636
PAY_AMT1                      6636
PAY_AMT2                      6636
PAY_AMT3                      6636
PAY_AMT4                      6636
PAY_AMT5                      6636
PAY_AMT6                      6636
default.payment.next.month    6636
dtype: int64
```

In [86]:
```
#seaborn plt.subplot for the use of heatmap visualizations
fig, ax = plt.subplots(figsize = (20,15))
corr = ccdf.corr().round(2)
sns.heatmap(corr, annot = True, ax=ax, vmin=-1, vmax=1, center=0, cmap='YlOrRd')
```

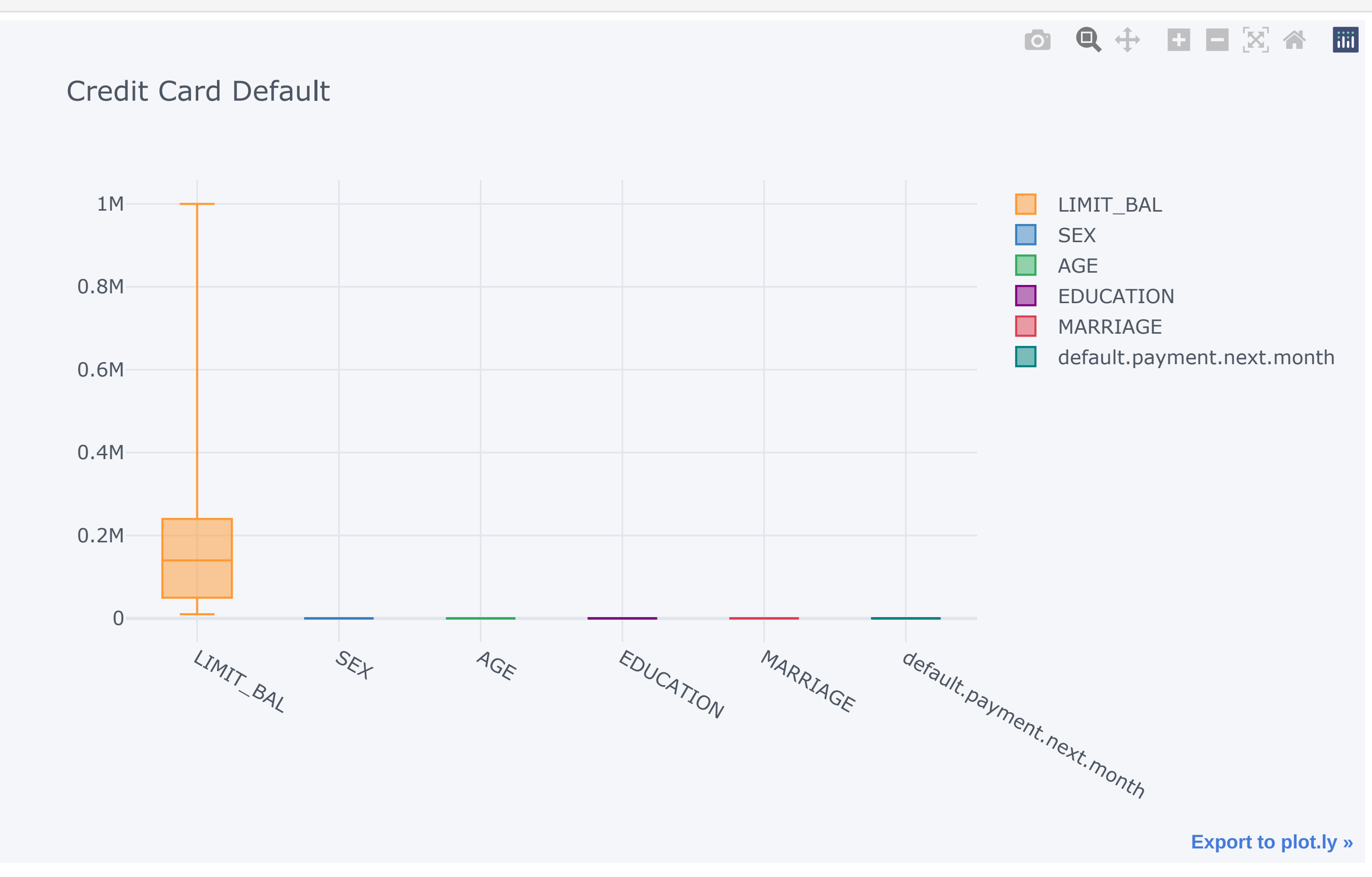
Out[86]:
```
<AxesSubplot:>
```



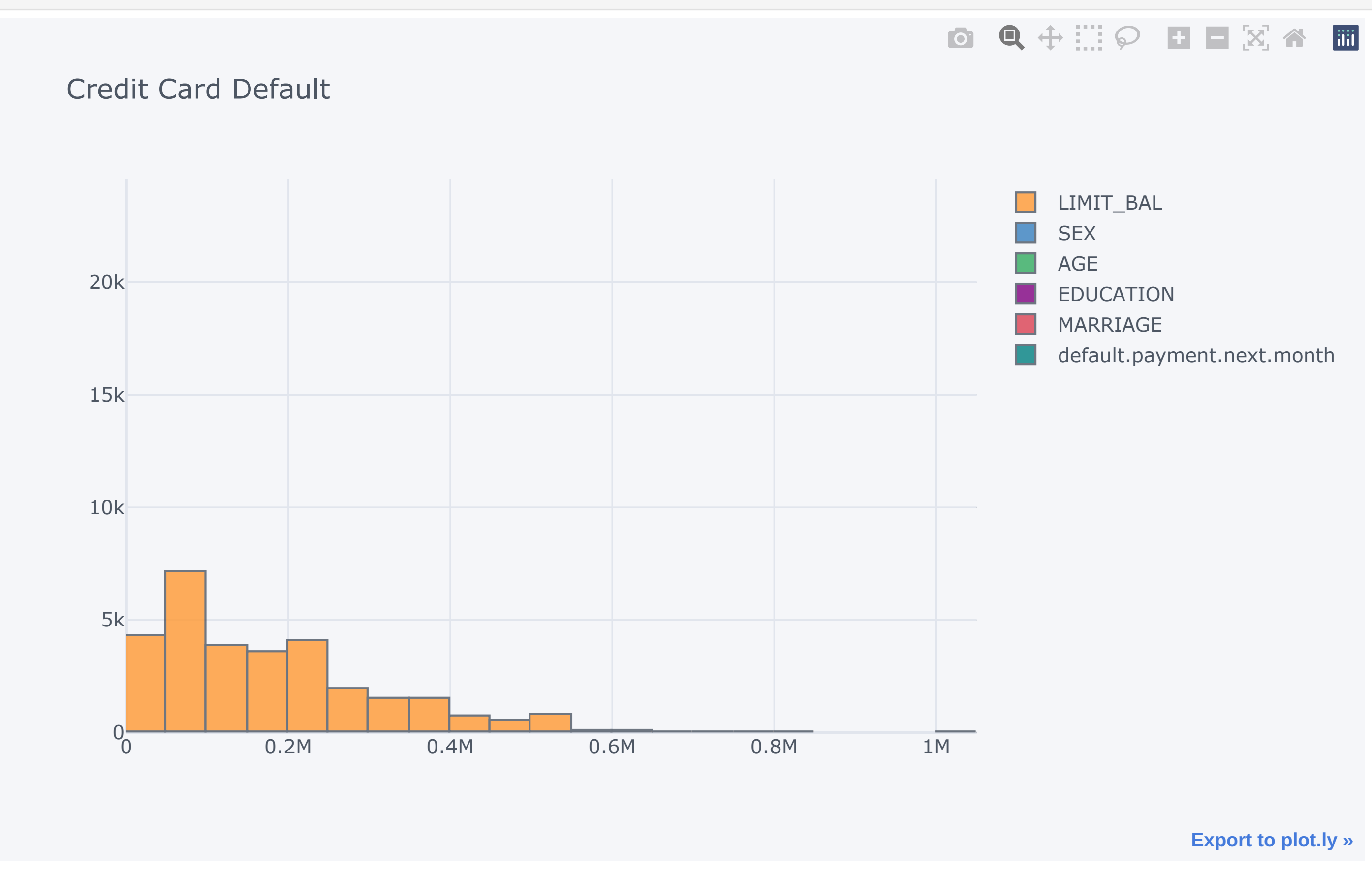
In [3]:
```
#this import assists with cufflinks
from plotly import __version__
from plotly.offline import download_plotlyjs, init_notebook_mode, plot, iplot

print(__version__) # requires version >= 1.9.0

import cufflinks as cf

init_notebook_mode(connected=True)

cf.go_offline()
```

```
5.7.0
```

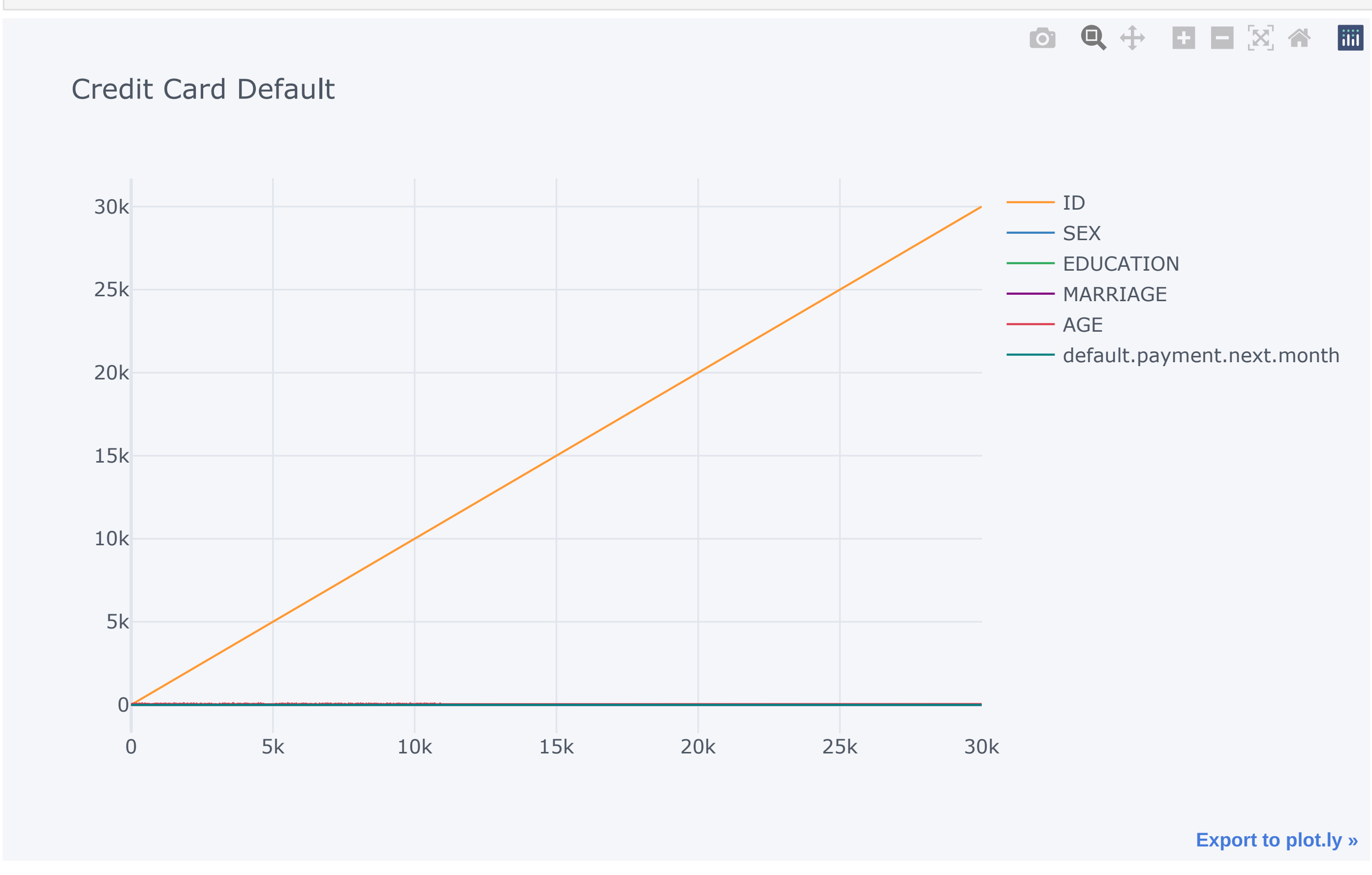
In [8]:
```
ccdf[['LIMIT_BAL','SEX','AGE','EDUCATION','MARRIAGE','default.payment.next.month']].iplot(kind='box',title='Cre
```



In [9]:
```
#df['A'].iplot(kind='hist',bins=25)
ccdf[['LIMIT_BAL','SEX','AGE','EDUCATION','MARRIAGE','default.payment.next.month']].iplot(kind='hist', bins=25,
```



In [10]:
```
ccdf[['ID','SEX','EDUCATION','MARRIAGE','AGE','default.payment.next.month']].iplot(kind='scatter',title='Credit
```



Machine Learning with Python

In [13]:
```
from sklearn.metrics import classification_report,confusion_matrix
```

In [14]:
```
from sklearn.model_selection import train_test_split
```

In [15]:
```
X = ccdf.drop('default.payment.next.month',axis=1)
y = ccdf['default.payment.next.month']
```

In [16]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.30, random_state=115)
```

Random Forest model with sklearn

In [17]:
```
#random_forest_model

from sklearn.ensemble import RandomForestClassifier
rfc = RandomForestClassifier(n_estimators=100)
rfc.fit(X_train, y_train)
```

Out[17]:
```
RandomForestClassifier()
```

In [18]:
```
rfc_pred = rfc.predict(X_test)
```

Random Forest Matrix - RFC confusion_matrix

In [41]:
```
print(confusion_matrix(y_test,rfc_pred))
```

```
[[6653  390]
 [1241  716]]
```

Random Forest Classification Report

In [20]:
```
print(classification_report(y_test,rfc_pred))
```

```
              precision    recall  f1-score   support

           0       0.84      0.94      0.89      7043
           1       0.65      0.37      0.47      1957

    accuracy                           0.82      9000
   macro avg       0.75      0.66      0.68      9000
weighted avg       0.80      0.82      0.80      9000
```

In [92]:
```
from IPython import display
display.Image("/home/harlohutch77/pictures/cc.gif",height='300', width='500')
```

Out[92]:
```
<IPython.core.display.Image object>
```